Time Is the Baseline to Beat: Lead Time over a Timer as the Test for VLA Failure Monitors

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—On RoboCasa365, a failed vision-language-action (VLA) rollout is one that reaches its time limit. A runtime failure monitor therefore competes with a timer that alarms at a fixed fraction of that limit. Calibrated at a 5% false-positive rate (FPR), such a timer flags every one of 380 prospective failures, about 12 policy inferences before the timeout. Recall cannot separate a learned monitor from this timer; only lead time can. We state this as a testable hypothesis: at a matched calibrated FPR, alarming when *either* a policy-internal score or the timer crosses its threshold warns earlier than the timer and than any combination of time signals alone. We give a split-conformal calibration of this two-threshold rule with a finite-sample FPR guarantee. On one VLA and 38 tasks, fusing the SAFE failure score with a single timer more than doubles the mean warning lead (0.090 to 0.207 of the time limit, exploratory), but most of that gain is task duration: a per-task timer, which alarms past the 95th percentile of each task's development success durations, nearly closes the gap without reading the policy. A preregistered confirmatory test on a fresh cohort of 760 rollouts, with a new calibration cohort, finds a small real gain over this per-task timer: 0.255 against 0.225 of the time limit (gain 0.030, 95% CI 0.007–0.093) at 0.061 FPR. Noise and within-task resampled scores fused the same way gain at most 0.009. The gain is above zero but below the margin of 0.05 we declared in advance, so the primary hypothesis is not supported. Stage 2 tests a second VLA, measures lead from the moment a failure becomes unrecoverable, and asks whether earlier alarms raise task success within a fixed time budget when they trigger a reset and retry.**

I. Introduction

Runtime monitors for VLA policies read the policy's hidden state and predict eventual failure [8], or score how unusual that state is relative to successful rollouts [27, 19]. They are compared with each other, but not with what the robot knows for free: how long the attempt has been running.

In RoboCasa365, as in most simulated manipulation benchmarks, an episode ends at first success or at a time limit, and in our test cohort every one of 380 failed rollouts ran to its limit. A timer at 90% of the limit therefore flags all of them at 4.7% FPR. Against it, a monitor can add only one thing: an earlier alarm at the same false-alarm rate, which leaves time to stop, reset or ask for help. We make this lead time the test, with three contributions.

1) **A testable hypothesis and endpoint.** We define the warning lead of an alarm as the fraction of the time limit left when it fires (zero for a missed failure) and state four hypotheses with decision rules fixed before the confirmatory data were analysed (Sec. IV).
2) **Calibrated OR-fusion with a guarantee.** We alarm when either a policy-internal score or the timer crosses its threshold, and calibrate both thresholds along a nested path with a split-conformal rule that bounds the FPR on a new successful rollout by α in finite samples (Proposition 1).
3) **Exploratory and confirmatory evidence.** On an opened prospective cohort, SAFE fused with a single timer more than doubles its mean lead, but a per-task timer that never reads the policy nearly matches it (Sec. VI). A preregistered test on a fresh cohort finds that the policy adds 0.030 of the time limit over the per-task timer, above zero but below our declared margin of 0.05 (Sec. VII).

The result is a small policy signal, not a useful one by our own standard: on timeout-defined benchmarks, policy internals add little to duration statistics, and monitors should be compared with a per-task timer and evaluated on failures that end early.

II. Related Work

Failure detection from policy internals. SAFE learns a failure score from a VLA's last-layer features and thresholds it with a functional conformal band [8]; we reuse its implementation. Follow-ups change the training data [18], labels [17, 14], features [28, 31] or policy [22]. None reports an elapsed-time detector. SAFE removes the duration shortcut by truncating rollouts to equal length [8], which a deployed monitor cannot do. Duration is a shortcut feature [7], but its causal form is legitimate evidence online, so we make it the baseline.

Action-space and success-only detectors. STAC [1] and VLA-FAIL [20] compare overlapping action chunks, and ActProbe [9] feeds chunk consistency and magnitude to a task-conditioned LSTM, beating internal-feature baselines on RoboCasa. Our policy executes each chunk in full, so overlap signals are undefined; chunk magnitude and a second flow sample are Stage 2 baselines. A task-conditioned recurrent detector can also learn how long each task takes, which our timers make explicit. FAIL-Detect [27] and FIPER [19] score inputs, outputs or embeddings against successful rollouts only. FIPER alarms when two conformal scores *both* exceed their thresholds, which suppresses false alarms; we alarm when *either* crosses, because the policy score and the timer catch failures at different times, and calibrate the pair jointly.

World-model verifiers and VLM monitors. Check-VLA [12] is the closest work on our benchmark. It checks

TABLE I
UNITS OF TIME, WITH THE FAILED *OPENCABINET* ROLLOUT OF FIG. 1 AS AN EXAMPLE.

Unit	Definition	Example
Control step	one simulator action, 20 Hz (50 ms)	1,050
Policy inference t	one 16-step action chunk (0.8 s)	66
Time limit T_k, H_k	official limit, control steps; $H_k = \lceil T_k/16 \rceil$ in inferences	1,050; 66
Video frame	every 2nd control step, shown at 20 fps	526
Elapsed fraction	t/H_k (SAFE alarm)	30/66
Warning lead ℓ	$(H_k - T)/H_k$ at alarm T	36/66 = 0.55

each action chunk against an action-conditioned world model on RoboCasa365, calibrates a conformal threshold to a 5% episode-level rate of unnecessary interventions, and repairs the chunk with the same VLA. Foresight [24] uses world-model latents. Neither compares against elapsed time, and either risk could be fused with a timer as we fuse SAFE. LLM and VLM monitors [1, 13, 21, 30, 6] are slower and complementary.

Time, calibration and intervention. VLA confidence drifts over a task [32]. Our time-risk baseline is a per-task survival-style estimate [10, 5], and lead time at a fixed false-alarm budget is the accuracy–earliness trade-off of early time-series classification [26]. Split-conformal thresholds give finite-sample false-positive control under exchangeability [3]; we extend the construction to a two-signal OR rule (Sec. V-C). RESCUE [2] finds that good detection scores do not guarantee lower intervention cost, and CheckVLA and FLARE [29] couple detection to repair. Hence our closed-loop Stage 2 test: an earlier alarm matters only if acting on it raises success.

III. Problem Setting

Time. We measure time in four units (Table I). The simulator advances in *control steps* at 20 Hz, RoboCasa's default control rate, which our collection code does not change. The policy is queried once per 16-step action chunk and executes the whole chunk, so one *policy inference* covers 16 control steps, or 0.8 s. Inferences are our clock: $t = 1, 2, \ldots$ counts them, and a monitor can change its decision only at an inference. The official time limit of task k is T_k control steps; in inferences it is $H_k = \lceil T_k/16 \rceil$, between 29 and 272 for our tasks (22.5 to 217.5 s). Videos keep every second control step (10 Hz) and play at 20 frames per second, twice real time. We use them only for figures. Every fraction of time in this paper, such as t/H_k, the timer fraction c and the lead ℓ, is a fraction of H_k.

Monitoring. A monitor watches a rollout of a known task k and decides at each inference t whether to alarm, using only the policy features so far, t, the task and H_k. The first crossing is the alarm T (in inferences). FPR is the fraction of successful rollouts with any alarm. A rollout of length L inferences ends at success or at H_k; for a failure ($L = H_k$) the *warning lead* is $\ell = (L - T)/H_k$, or 0 if missed, and an alarm at the final inference counts with $\ell = 0$. The mean lead $\bar{\ell}$ combines recall and earliness. Detectors are compared at operating points fixed on a calibration cohort under FPR $\leq \alpha = 0.05$. Confidence intervals come from a paired bootstrap that resamples tasks, then rollouts within each task (2,000 replicates).

IV. Hypotheses

We test four hypotheses. H1 and H2 were evaluated on a fresh confirmatory cohort with the decision rules below, registered as an amendment before any timer or lead quantity was computed on that cohort (Sec. VII); H3 and H4 are the Stage 2 extensions (Sec. VIII).

H1 (lead time, primary). At thresholds calibrated for FPR $\leq$ 0.05, OR-fusing a policy-internal score with the per-task timer (Sec. V), the strongest time-only monitor we know, increases the mean warning lead over that timer alone. *Supported* if the paired bootstrap 95% CI of $\Delta\bar{\ell}$ lies above zero, the point gain is at least 0.05 of the time limit, and the fused test FPR is at most 0.075 with a Wilson 95% upper bound at most 0.10. As a secondary check, the fused detector must also beat a timer whose fraction is tuned *on the test cohort* to the fused detector's test FPR, which favours the timer.

H2 (the gain needs policy evidence). Fusions that contain no policy-internal evidence (Gaussian noise, or SAFE scores resampled within task, each fused with the per-task timer) gain less than 0.05 of the time limit over the per-task timer.

H3 (generality). H1 holds for a second VLA with a different architecture.

H4 (utility). When an alarm triggers a reset and retry, the OR-fused monitor yields a higher task success rate than the timer within the same per-task time budget.

Only H1 and H2 are confirmatory; every other interval in this paper is descriptive. On the opened data the H1 gain was 0.007, so we expected H1 to fail. It did, but with a gain of 0.030 whose CI excludes zero (Sec. VII).

V. Method

A. Signals

Policy score. At each inference we take the final-layer action-token hidden state of the policy's action head (1,024 dimensions). The SAFE MLP [8] maps it to a failure score $u_t^{(j)}$ for each of three training seeds j. With per-task location $m_k^{(j)}$ and scale $\sigma_k^{(j)}$ estimated on development rollouts, the policy score is

$$s_t = \frac{1}{3}\sum_{j=1}^{3} \frac{\max_{t' \leq t} u_{t'}^{(j)} - m_k^{(j)}}{\sigma_k^{(j)}}. \tag{1}$$

The running maximum makes s_t monotone and does not change when a threshold is first crossed. Success-only alternatives (Mahalanobis [11], kNN [23], PCA-KMeans and RND [4]) are fitted on successful development rollouts and aggregated the same way.

Timers. The global timer alarms once $t \geq c\,H_k$ for a fraction $c \in (0, 1]$ shared by all tasks; it uses only the task's time limit. The per-task timer uses $c_k = \min(1, \rho_k + \delta)$, where ρ_k is the 95th percentile of task k's development success

durations as fractions of H_k and the shared margin δ is set by Proposition 1 on calibration successes. It needs successful rollouts of each task, but no failures and no policy features. We fixed the 95th percentile after auditing the opened cohort, so we report 0.75, 0.90 and 1.0 (the longest success) as sensitivity.

Time risk (baseline). The survival-style alternative estimates $r_t = \Pr(\text{failure} \mid k, \text{still running at } t)$ from development rollouts: with n_t of them still running at t and f_t of those failing, it fits an increasing isotonic regression in t/H_k to $(f_t + 0.5)/(n_t + 1)$, weighted by n_t. Because every failure runs to H_k, r_t is a task-specific timer; its smoothing keeps it below a high threshold in tasks with few development failures, which is why it misses failures the plain timer catches.

B. OR-fusion

The fused monitor alarms at

$$T = \min\{\, t : s_t > \tau_s \text{ or } t \geq c\, H_k \,\}. \tag{2}$$

Neither signal is restricted in time: the policy score may alarm early, and the timer guarantees an alarm on every failure, at $c\, H_k$ at the latest. The lead gain over the timer comes only from failures on which the policy score crosses first.

C. Calibration with a finite-sample FPR guarantee

A successful rollout i of length L_i (in inferences) raises a false alarm under (τ_s, c) if and only if $M_i = \max_{t \leq L_i} s_t > \tau_s$ or $\rho_i = L_i/H_{k_i} \geq c$, so its false alarm depends only on the pair (M_i, ρ_i). Fix in advance a path $\gamma(\lambda) = (\tau_s(\lambda), c(\lambda))$, $\lambda \in [0, 1]$, with both coordinates continuous and nondecreasing, so that a larger λ is more conservative and $\gamma(1)$ never alarms on a success. For each calibration success define

$$V_i = \min\{\lambda : M_i \leq \tau_s(\lambda) \text{ and } \rho_i < c(\lambda)\}, \tag{3}$$

the least conservative point on the path at which rollout i stays silent. Because the path is monotone, rollout i is silent at λ exactly when $\lambda \geq V_i$.[1]

Proposition 1: Let n calibration successes and a new successful rollout be exchangeable, and let the path γ not depend on the calibration successes. Set $\hat{\lambda} = V_{(\lceil (n+1)(1-\alpha) \rceil)}$, the $\lceil (n+1)(1-\alpha) \rceil$-th smallest V_i (or $\hat{\lambda} = 1$ if that rank exceeds n). Then the OR-fused monitor at $\gamma(\hat{\lambda})$ raises a false alarm on the new success with probability at most α.

Proof: The new rollout alarms at $\hat{\lambda}$ if and only if $V_{n+1} > \hat{\lambda} = V_{(r)}$ with $r = \lceil (n+1)(1-\alpha) \rceil$. By exchangeability, the rank of V_{n+1} among $V_1, \ldots, V_{n+1}$ is uniform on $\{1, \ldots, n+1\}$ (ties broken at random), so $\Pr(V_{n+1} > V_{(r)}) \leq 1 - r/(n+1) \leq \alpha$; this is the split-conformal quantile lemma [3] applied to the scalar V. ∎

The path may use any data except calibration successes, such as development rollouts or calibration *failures*. The guarantee is marginal over tasks; a per-task version needs 19 calibration successes per task. The deployed FPR is Beta-distributed given calibration: it exceeds 0.075 with probability 0.065 for $n = 114$, but 0.021 for $n = 380$.

[1]The strict $\rho_i < c(\lambda)$ makes the set right-open; taking c on a finite grid, as the timer's resolution of one inference does anyway, makes the minimum attained.

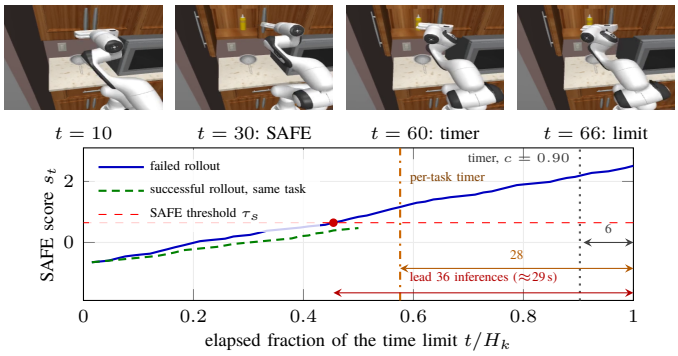


Fig. 1. One failed *OpenCabinet* rollout from the prospective test (chosen to illustrate; the median SAFE-first lead is smaller). The SAFE score crosses its calibrated threshold at $t = 30$ of $H_k = 66$ inferences, 36 inferences before the time limit; the timer alarms at $t = 60$, and the per-task timer, which never reads the policy, at $t = 38$. Dashed: the longest of the 9 (out of 10) successful test rollouts of this task that stay below the threshold; it ends at $t = 33$.

The preliminary results below use the simpler joint search of the original protocol: among all pairs of observed calibration maxima with calibration FPR $\leq \alpha$, keep the pair with the largest mean lead (highest recall for the time-risk fusions). Selecting the best of many non-nested pairs on the same 114 successes has no finite-sample guarantee and is optimistic, which matches the test FPRs of 0.058–0.068 we observe for two-threshold detectors against 0.044 on calibration. The per-task timer rows already use Proposition 1, with the path $\delta = \lambda$, $\tau_s = u + s\lambda$ and (u, s) chosen on calibration failures; the confirmatory test calibrates both the per-task timer and its SAFE fusion this way.

VI. Preliminary Evidence

Setup. We use the released Xiaomi-Robotics-1 RoboCasa365 checkpoint [25], which executes each 16-step action chunk in full, on the 38 official RoboCasa365 tasks [15, 16] with published success between 5 and 45 of 50 and enough failures (10 atomic, 28 composite; failures last 29–272 inferences). Disjoint cohorts serve one purpose each: development (1,824 rollouts) fits SAFE and the time risk; calibration (3 successes and 3 failures per task; 454 rollouts including those past the quota) sets thresholds; the prospective test (10 per class per task) is evaluated once; a confirmatory cohort of the same size is used only in Sec. VII. SAFE, its normalization and the time risk were frozen before the test was collected; the timers and fused detectors were added later, with thresholds from calibration only, so the results below are exploratory.

The timer is the baseline to beat. The timer ($c = 0.903$) alarms on all 380 test failures at 0.047 FPR with a fixed lead of 0.090 (Table II). The time-risk curve misses 140 failures, all in 14 tasks whose smoothed curve never reaches its threshold, so recall gains measured against it (to 0.90–0.96 for SAFE and four success-only scores) vanish here.

SAFE adds lead over a single timer. SAFE + timer raises the mean lead to 0.207 (paired gain 0.117, CI 0.060–0.178;

TABLE II

PROSPECTIVE TEST (380 SUCCESSES, 380 FAILURES), THRESHOLDS FROM CALIBRATION AT FPR ≤ 0.05. $\bar{\ell}$ = MEAN WARNING LEAD AS A FRACTION OF THE TIME LIMIT (MISS = 0); MED = MEDIAN LEAD IN INFERENCES OVER DETECTED FAILURES; @.75 = FAILURES ALARMED BY $0.75\,H_k$. *MEAN OVER 20 DRAWS (RANGE OF $\bar{\ell}$ 0.088–0.134). †95TH PERCENTILE OF DEVELOPMENT SUCCESS DURATIONS, MARGIN BY PROPOSITION 1 ON THE 114 CALIBRATION SUCCESSES.

Detector	Recall	FPR	$\bar{\ell}$	Med	@.75
Timer ($c = 0.903$)	1.000	0.047	0.090	11.5	0.000
Time risk	0.632	0.047	0.120	16.5	0.211
Time risk + timer	1.000	0.050	0.125	11.0	0.105
Noise + timer*	1.000	0.066	0.101	–	–
SAFE only	0.566	0.039	0.190	23.0	0.421
SAFE + time risk	0.955	0.058	0.184	12.0	0.311
Per-task timer†	1.000	0.045	0.195	13.5	0.342
SAFE + per-task timer†	1.000	0.042	0.202	14.0	0.389
SAFE + timer	1.000	0.058	0.207	10.0	0.421

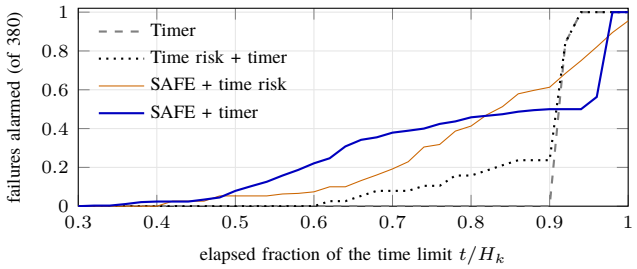


Fig. 2. Fraction of the 380 test failures alarmed by each elapsed fraction of the time limit. SAFE + timer catches half of the failures early and the rest at its timer fallback ($c = 0.96$).

20.2 against 11.7 inferences) and flags 42% of failures by $0.75\,H_k$ (Figs. 1 and 2), at 0.058 test FPR. Noise fused with the timer reaches 0.101, and the time risk with the timer 0.125. Among rollouts still running at 10, 25, 50 and 75% of the limit, SAFE's within-task ROC-AUC is 0.55–0.60, against 0.41–0.51 for noise. On test-cohort frontiers, OR and AND fusion reach the same lead at 5% FPR (0.208 and 0.211); OR keeps the timer's guaranteed alarm.

A per-task timer removes the gain (H1, exploratory). The per-task timer reaches $\bar{\ell} = 0.195$ at 0.045 FPR without reading the policy. SAFE + timer leads it by an insignificant 0.012 (CI -0.044 to 0.063), and by -0.003 against a per-task timer tuned on test to the same FPR. SAFE + per-task timer reaches 0.202, a gain of 0.007 (CI -0.003 to 0.019), far below the 0.05 margin of H1. The two effects cancel across tasks: in the 14 tasks where the time risk never fires, SAFE + timer's mean lead is 0.405 against 0.221 for the per-task timer; in the other 24 it is 0.092 against 0.180.

No early warning yet. No detector flags more than 8% of failures by half of the limit; whether a late warning is usable is H4.

VII. CONFIRMATORY TEST OF H1 AND H2

Protocol. A fresh test cohort (new seeds, 10 successes and 10 failures per task) was collected with SAFE frozen and preregistered for the recall endpoint against the time risk.

TABLE III

CONFIRMATORY TEST (380 SUCCESSES, 380 FAILURES). P = PER-TASK TIMER ($q = 0.95$). GAINS AGAINST P; THE H1 INTERVAL IS THE REGISTERED DECISION BOOTSTRAP, THE OTHERS RESAMPLE THE TEST COHORT ONLY. *MEAN OVER 20 DRAWS (MAXIMUM IN BRACKETS). †THRESHOLDS FROZEN FROM THE ORIGINAL PROTOCOL.

Detector	Recall	FPR	$\bar{\ell}$	Gain vs P
Global timer	1.000	0.024	0.123	-0.102 (-0.147, -0.059)
SAFE + time risk†	0.955	0.039	0.190	-0.035 (-0.066, -0.007)
SAFE + global timer	1.000	0.047	0.215	-0.011 (-0.068, 0.045)
P	1.000	0.055	0.225	–
Noise + P*	–	–	–	0.002 [0.009]
Resampled SAFE + P*	–	–	–	0.002 [0.007]
SAFE + P (H1)	1.000	0.061	**0.255**	**0.030** (0.007, 0.093)

Before any timer or lead quantity was computed on it, an amendment made the per-task timer P ($q = 0.95$) the primary comparator and $\Delta\bar{\ell}$ the primary endpoint, with the rules of Sec. IV; the cohort's recall and FPR were already known. A new calibration cohort of the same design ($n = 380$ successes, disjoint seeds) sets the thresholds of P and SAFE + P by Proposition 1. The decision interval resamples calibration and test together and refits the thresholds in each of 2,000 replicates. Every test failure again ran to H_k.

H1 is not supported, but the gain is not zero. SAFE + P leads P by 0.030 of the time limit (Table III). Its decision CI lies above zero and its test FPR (0.061, Wilson 95% upper bound 0.089) meets the FPR rules, but the point gain is below the 0.05 margin, so H1 fails. The CI's upper end (0.093) does not rule out a gain above the margin: the test missed the margin; it did not show the gain is negligible. The opened-cohort estimate (0.007) understated it.

H2 is supported. Noise and within-task resampled SAFE scores, fused with P and calibrated the same way, gain at most 0.009 over P in 20 draws each. What SAFE adds is policy evidence, not an artifact of fusing any score with a timer.

Task duration carries most of the lead. P alone leads the global timer by 0.102, and SAFE + global timer reaches about what P reaches without reading the policy. The original fused design, SAFE + time risk, which beat the time risk on recall in this same cohort, gives 0.035 *less* lead than P. Part of the H1 gain is FPR budget: a per-task timer tuned on the test cohort to SAFE + P's FPR leaves a gain of only 0.015 (CI -0.007 to 0.041), and a global timer tuned the same way 0.024 (CI -0.022 to 0.074), so the secondary check of H1 also fails. The gain is stable across the timer quantile (0.026, 0.028 and 0.030 for $q = 0.75$, 0.90 and 0.95) and grows to 0.074 at $q = 1$, where P itself is weaker (0.164).

Atomic tasks carry the gain (descriptive). On the 10 atomic tasks SAFE + P gains 0.081 (0.372 against 0.291); on the 28 composite tasks, 0.012. This matches the atomic advantage of SAFE in our earlier analyses, but it is a subgroup result and FPR is not controlled per stratum.

VIII. STAGE 2 PLAN

E1: Confirmatory test of H1 and H2. Done (Sec. VII).[PENDING: Decide whether a registered atomic-

only test (Amendment 2) belongs in Stage 2; the exploratory dose-response analysis on the opened cohorts gates it.]

E2: Second policy (H3). We repeat the timer, time-risk, SAFE and fused rows with π_0 or RLDX-1 on the same 38 tasks, reusing saved rollouts where hidden states were recorded. A policy with overlapping chunks also lets us run chunk-consistency monitors [1, 9].

E3: Closed-loop reset and retry (H4). On 8 to 10 tasks with mid-range success, each episode gets a budget of $2H_k$ inferences. Under each monitor (none, timer, SAFE + timer), an alarm ends the attempt, resets the scene to its initial state and starts a new attempt while budget remains; without a monitor, the retry starts at the timeout. We compare success within the budget, paired by environment seed, and count successes aborted by false alarms. The gain is bounded by the failure rate times the retry success rate times the fraction of failures whose extra lead leaves room for a retry; a null result would bound the value of any such monitor on this benchmark.

E4: Stronger baselines and checks. (i) Action-only monitors that need no hidden state: chunk magnitude and the disagreement of a second flow sample. (ii) Richer time baselines: a stall detector on end-effector motion and a discrete-time hazard model on simple observables, so that "time" means the best signal available without the policy's internals. (iii) OR- and AND-fusion compared on full recall–FPR curves rather than single points. (iv) SAFE faithfulness checks and precision at deployment prevalence. (v) An unseen-task split, holding tasks out of SAFE training and normalization; the timer needs only H_k and transfers as is.

E5: Failures that end early. Every failure here ends at the limit. We will label, on a subset, when a failure became unrecoverable (a dropped or displaced object), and report lead relative to that moment, since these are the failures a deployed robot most needs to catch early.

IX. Limitations

All results come from one policy in simulation, in a shadow evaluation where monitors never intervened. The exploratory cohort was opened before the timers and fused detectors were defined, and the confirmatory endpoint was registered after that cohort's recall and FPR were known. SAFE + P ran at a higher test FPR than P (0.061 against 0.055), and part of its gain is that budget. The timers and per-task normalization assume a known task and time limit, and the policy score needs white-box access and development rollouts of each task; on the confirmatory cohort, that access bought 0.030 of lead over the per-task timer. Test cohorts are balanced by quota, so recall and FPR are per class, not at deployment prevalence.

X. Conclusion

When failure means timeout, a calibrated timer catches every failure, and the question for a learned monitor becomes how much earlier it can warn at the same false-alarm rate. On one VLA and 38 tasks, fusing a hidden-state failure score with a single timer more than doubled the mean warning lead, but a per-task timer built from development success durations nearly matched it without reading the policy. On a fresh preregistered cohort, the hidden state added 0.030 of the time limit on top of that timer: a real signal, below the margin we declared in advance. Stage 2 tests a second policy and failures that become unrecoverable before the timeout, and asks the question that decides the value of any such monitor: whether the extra warning lets a robot recover more often within the same time.

References

[1] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024.

[2] David Ahmedt-Aristizabal, Abdelwahed Khamis, Mohammad Ali Armin, and Lars Petersson. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors, 2026. arXiv:2610.13516.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[4] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.

[5] David R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2):187–220, 1972.

[6] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2 (11):665–673, 2020.

[8] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[9] Bingjia Huang, Xiangyu Li, Xiang Wang, Liang Mi, Zixu Hao, Weijun Wang, Hao Wu, Kun Li, Yunxin Liu, and Ting Cao. ActProbe: Action-space probe for early failure detection of generative robot policies, 2026. arXiv:2606.08508.

[10] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.

[11] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-

distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[12] Yushan Liu, Peibo Sun, Xintao Chao, Zhenyang Yang, Yifan Xie, Lingfeng Zhang, Shoujie Li, Chenyu Tang, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. CheckVLA: Execution-time verification with action-conditioned world model for long-horizon mobile manipulation, 2026. arXiv:2607.26789.

[13] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.

[14] Jie Ma, Zongxi Liu, and Yi Zhu. FailureSpot: Label-efficient timestamp-level failure detection for vision-language-action models, 2026. arXiv:2609.04277.

[15] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[16] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.

[17] Seongheon Park, Wendi Li, Changdae Oh, Min-Hsuan Yeh, Zsolt Kira, Michael Hagenow, and Sharon Li. Hide-and-seek in trajectories: Discovering failure signals for VLA runtime monitoring. In *ICML Workshop on Failure Modes in Agentic AI*, 2026. arXiv:2605.30834.

[18] Harshitha Rajaprakash, Aditeya Prajapati, Rong Xue, Abrar Anwar, and Jesse Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration, 2026. arXiv:2608.04246.

[19] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela P. Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[20] Florian Seligmann, Emiliyan Gospodinov, Enes Ulas Dincer, and Gerhard Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models, 2026. arXiv:2606.21386.

[21] Rohan Sinha, Amine Elhafsi, Christopher Agia, Matthew Foutter, Edward Schmerling, and Marco Pavone. Real-time anomaly detection and reactive planning with large language models. In *Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.114.

[22] Michael Soddy, Karthikeyan Ganesan, Roman Stemp, Ved Deo, and Nicholas Babey. Predictable before action, resistant to correction: Failure signals in vision-language-action models. In *NeurIPS Workshop on From Pretrained Representations to Acting Agents (PTA)*, 2026.

[23] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *International Conference on Machine Learning (ICML)*, 2022.

[24] Boyang Wang, Haoran Zhang, Mengdi Wang, Odest Chadwicke Jenkins, Xuhui Kang, Yen-Ling Kuo, Yifu Lu, and Zezhou Cheng. Foresight: Failure detection for long-horizon robotic manipulation with action-conditioned world model latents, 2026. arXiv:2606.23085.

[25] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories, 2026. arXiv:2607.15330.

[26] Zhengzheng Xing, Jian Pei, and Philip S. Yu. Early classification on time series. *Knowledge and Information Systems*, 31(1):105–127, 2012. doi: 10.1007/s10115-011-0400-x.

[27] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025.

[28] Jinghan Yang, Yunchao Zhang, Wang Yuan, Haolun Wan, Jiaming Zhang, Zhengyang Hu, and Yanchao Yang. Tri-Info: Generalizable, interpretable failure prediction for VLA models via information theory, 2026. arXiv:2606.19998.

[29] Ganlong Zhao, Zijia Tang, Xingping Chen, Zhanghui Kuang, Ye Tian, and Guanbin Li. FLARE: A failure-aware framework for autonomous correction and recovery in visual-language robotic manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22391–22401, 2026.

[30] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6919–6929, 2025.

[31] Kaiwen Zhu, Dongfang Liu, and Liangkai Liu. VLA-Scope: Shift-aware failure prediction for vision-language-action models, 2026. arXiv:2609.21246.

[32] Tom Zollo and Richard Zemel. Confidence calibration in vision-language-action models. In *ICML Workshop on Statistical Frameworks for Uncertainty in Agentic Systems*, 2026. arXiv:2507.17383.